RAG_NCERT_Biology_Teacher — Complete Project Guide (Beginner Friendly)

> This is a teaching document. `Readme.md` is the short project plan/checklist. **This file** is the long-form guide that explains, from absolute zero, every command we ran, every file we created, *why* we created it, and what it does. It is updated every time we finish a step — read it top to bottom the first time, then jump to a section later when you forget why something exists.

---

0. What are we building, in one paragraph

We are building a **RAG (Retrieval-Augmented Generation) system**. In plain words: normally an AI chatbot only knows what it was trained on and can "hallucinate" (make up wrong facts). A RAG system fixes this by first **searching your own documents** for the relevant text, and then telling the AI: "here are the exact paragraphs from the textbook — now use ONLY these to answer the question." We are pointing this at the **NCERT Class 12 Biology textbook**, and making the AI behave like a **teacher** explaining concepts to a student, instead of a generic chatbot.

---

1. Starting from absolute zero

1.1 The empty folder

Everything starts from one empty folder on your computer:

```
D:\GenAI\RAG\Projects\RAG_NCERT_Biology_Teacher
```

This is called the **project's working directory** (or "root"). Every file we create, every command we run, is relative to this one folder — it's the home base for this project and nothing outside it is touched.

1.2 Opening a terminal in that folder

To *do* anything (install libraries, run Python code), we need a **terminal** (command line) that is "standing inside" that folder. All the commands below assume the terminal's current location is that folder. In VS Code, this is the integrated terminal panel; it opens already pointed at your project folder.

1.3 Turning the folder into a git repository

```
git init
```

What this does: creates a hidden `.git/` folder that turns your project folder into a **version-controlled repository**. From this point on, git can track every change you make to any file, and you can save "checkpoints" (commits) you can always go back to. We are only doing this *locally* for now — nothing is pushed/uploaded anywhere until you explicitly approve it.

---

2. Setting up the Python project with `uv`

2.1 Why `uv` instead of plain `pip`/`venv`

Every Python project needs: 1. Its own **isolated environment** (so its libraries don't conflict with other Python projects on your machine). 2. A **manifest** listing exactly which libraries (and versions) it needs.

`uv` is a modern all-in-one tool that does both, faster than the older `python -m venv` + `pip install` + `requirements.txt` combo, and it also writes an exact **lockfile** so the project installs identically every time.

2.2 The command we ran

```
uv init --app --no-readme --vcs none --name rag-ncert-biology-teacher .
```

- `--app` → this is an application (not a reusable library others `pip install`).
- `--no-readme` → don't auto-generate a `README.md` (we already have our own `Readme.md` with the project plan; this avoids `uv` overwriting it).
- `--vcs none` → don't run `git init` again (we already did that ourselves).
- `--name rag-ncert-biology-teacher` → internal project name (the project was later renamed from the initial placeholder `ncert-rag` to this more descriptive name — the Python package was renamed to match: `src/rag_ncert_biology_teacher/`).
- `.` → initialize in the **current folder** instead of creating a new subfolder.

2.3 What files this created, and why each one exists

File	Purpose
`pyproject.toml`	The project's manifest. Lists the project name, the Python version required, and (after Step 2.4 below) every library dependency. Think of it as the master ingredient list for this project.
`.python-version`	Pins which Python version (3.13) this project uses, so `uv` always picks the right interpreter.

`main.py` (uv created this, **we deleted it**)	`uv init` scaffolds a generic "Hello World" entry point by default. Our project's real entry point will be the Streamlit app (`app/Home.py`, coming in Step 9), so this placeholder wasn't needed.

2.4 Adding the actual libraries the project needs

```
uv add langchain langchain-community langchain-text-splitters chromadb \
 langchain-chroma pymupdf sentence-transformers transformers torch \
 Pillow streamlit deepeval python-dotenv
```

What this does:

- Downloads each library and every library *it* depends on.
- Creates `.venv/` — a private folder holding a full Python installation + all these libraries, used **only** by this project.
- Writes `uv.lock` — records the *exact* version of every single package installed (including indirect dependencies), so the environment is perfectly reproducible later (e.g. on another machine, or in deployment).
- Adds every package to the `dependencies = [...]` list inside `pyproject.toml`.

Why each library, in plain terms:

Library	What it's for
`langchain`, `langchain-community`, `langchain-text-splitters`	Orchestration toolkit — ready-made building blocks for RAG pipelines (splitting text, tracking indexed documents, connecting to vector databases, wiring retrieval + LLM together).
`chromadb`, `langchain-chroma`	**Chroma** is the vector database — where we store the "meaning fingerprints" (embeddings) of every chunk of the textbook, so we can later search by meaning instead of by exact keyword.
`pymupdf`	Opens PDF files and extracts text + embedded images page by page (Step 3).
`sentence-transformers`	Provides the embedding model — turns text into a vector (a list of numbers) that represents its meaning (Step 6).
`transformers`, `torch`	The machine-learning engine underneath — needed to run the image-captioning model (Step 4) and the embedding model locally, for free, without an API key.

`Pillow`	Basic image file handling (opening/resizing images) for the captioning step.
`streamlit`	Turns our Python scripts into an actual web page/UI you click around in — no HTML/JS needed.
`deepeval`	A testing framework specifically for RAG systems — scores whether the AI's answers are actually faithful to and relevant to the retrieved textbook content (Step 13).
`python-dotenv`	Reads secret values (like API keys) from a local `.env` file instead of hardcoding them in code.

2.5 How we actually *run* Python code in this project from now on

Because everything lives inside `.venv/`, we don't call `python` directly — we prefix commands with `uv run`, e.g.:

```
uv run python -m rag_ncert_biology_teacher.config
```

`uv run` makes sure the command executes *inside* this project's isolated environment, using exactly the library versions in `uv.lock`.

---

3. `.gitignore` — telling git what NOT to track

Why this file exists: not everything in a project folder should be saved into git history. Some things are:

- huge (the `.venv/` folder can be gigabytes),
- regenerable (delete `.venv/` and `uv sync` rebuilds it from `uv.lock`),
- or secret (`.env` holding API keys — this must **never** be committed).

`.gitignore` lists patterns for files git should simply pretend don't exist. Ours currently ignores: `.venv/`, `.env`, the large NCERT PDF/zip files under `data/raw_pdfs/`, everything under `data/extracted/` (regenerable output), the Chroma database folder, and editor/IDE clutter.

We deliberately still **track** `data/raw_pdfs/**/chapters.json` — it's tiny and documents which chapters *should* be there, even though the big PDFs themselves aren't committed.

---

4. `src/rag_ncert_biology_teacher/config.py` — the project's single source of truth for settings

Why this file exists: without it, every script would hardcode its own guess at "where are the PDFs?" or "where's the database?" — and the moment you moved a folder, ten files would break. Instead, every other module does:

```
from rag_ncert_biology_teacher.config import RAW_PDF_DIR, EXTRACTED_DIR, CHROMA_DIR
```

What's inside it:

- `PROJECT_ROOT`, `RAW_PDF_DIR`, `EXTRACTED_DIR`, `CHROMA_DIR` — absolute paths computed from this file's own location, so they work no matter where you run a command from.
- `EMBEDDING_MODEL_NAME`, `CAPTIONING_MODEL_NAME` — which pretrained models to use in later steps.
- `LLM_PROVIDER`, `LLM_MODEL` — read from environment variables (via `python-dotenv` + `os.getenv`), so the LLM provider/model can be swapped by editing `.env` instead of editing code.
- `ensure_directories()` — creates the data folders if they don't exist yet.
- The `if __name__ == "__main__":` block at the bottom prints out all the resolved paths — this is our "smoke test" to confirm the config loads correctly with no typos/errors.

---

5. Step 2 — Getting real data: downloading the NCERT textbook

Why: a RAG system needs *something* to retrieve from. We chose the real, current **NCERT Class 12 Biology** textbook (13 chapters, official NCERT government PDFs, free and public).

What we did: downloaded each chapter as its own PDF directly from `ncert.nic.in`, saved into:

```
data/raw_pdfs/class12_biology/
■■■ chapter_01.pdf   Sexual Reproduction in Flowering Plants
■■■ chapter_02.pdf   Human Reproduction
■■■ chapter_03.pdf   Reproductive Health
■■■ chapter_04.pdf   Principles of Inheritance and Variation
■■■ chapter_05.pdf   Molecular Basis of Inheritance
■■■ chapter_06.pdf   Evolution
■■■ chapter_07.pdf   Human Health and Disease
■■■ chapter_08.pdf   Microbes in Human Welfare
■■■ chapter_09.pdf   Biotechnology: Principles and Processes
■■■ chapter_10.pdf   Biotechnology and its Applications
■■■ chapter_11.pdf   Organisms and Populations
■■■ chapter_12.pdf   Ecosystem
■■■ chapter_13.pdf   Biodiversity and Conservation
■■■ chapters.json    a small manifest mapping each filename to its real title
```

Why `chapters.json`: filenames like `chapter_07.pdf` don't tell you what's *inside*. This manifest is our lookup table — later, when we attach metadata to search results ("this answer came from Chapter 7"), we read the real title from here instead of hardcoding it again somewhere else.

Why a `raw_pdfs` folder separate from `extracted`: this is a deliberate data-pipeline habit. `raw_pdfs/` is the original, untouched source — we never modify it. Every later step *reads* from `raw_pdfs/` and *writes*

its results into `extracted/`. If a later step has a bug, we fix the code and re-run it against the same raw files — we never have to re-download anything.

---

6. Step 3 — Document Loading: turning PDFs into text + images

Why this step exists: a PDF is a page-rendering format, not a database of text. Before we can search "explain photosynthesis" against this book, we need to physically pull out (a) the text on every page, and (b) every embedded image/diagram, as separate files.

File: `src/rag_ncert_biology_teacher/ingestion/pdf_loader.py`

Code walkthrough

```
@dataclass
class PageContent:
    page_number: int
    text: str
    image_paths: list[str] = field(default_factory=list)
```

A `dataclass` is a labeled container. Instead of a bare tuple like `(1, "some text", [...])` where you'd have to remember what each position means, this gives every piece of data a name: `.page_number`, `.text`, `.image_paths`. Every later step reads from objects shaped like this.

```
doc = fitz.open(pdf_path)
for page_index, page in enumerate(doc):
```

`fitz` is the import name for the **PyMuPDF** library. `fitz.open()` opens the PDF and lets us treat it like a list of pages. `enumerate` gives us both the page's position (`page_index`, starting at 0) and the page object.

```
text = page.get_text().strip()
```

Asks PyMuPDF: "give me every character drawn on this page, as a string."

```
for image_position, image_info in enumerate(page.get_images(full=True)):
    xref = image_info[0]
    base_image = doc.extract_image(xref)
```

`page.get_images()` doesn't give you image bytes directly — it gives a list of internal reference numbers (**xref** = "cross-reference", how the PDF format internally points at objects it stores). `doc.extract_image(xref)` then actually pulls out the raw image bytes and file type (`png`/`jpeg`) for that specific reference.

```
image_path.write_bytes(image_bytes)
```

Saves each image to its own file, e.g. `page_005_img_0.png`, inside a folder specific to that chapter — so chapter 3's images never mix with chapter 4's.

The `if __name__ == "__main__":` block runs only when this file is executed directly (not when imported by another file later). It processes one chosen chapter and prints a summary: page count, image count, a text preview, and a sample image path — this is how we *verify* the step actually worked, instead of just trusting the code compiles.

How to run it:

```
uv run python -m rag_ncert_biology_teacher.ingestion.pdf_loader class12_biology/chapter_03.pdf
```

---

7. What's coming next (updated as we build each one)

Step	What it does	Status
4. Image captioning	Turn extracted diagrams into text captions using an AI vision model, so figures become searchable too.	■ next
5. Chunking	Split each page's text into overlapping ~1000-character pieces with metadata (book/chapter/page) attached.	pending
6. Embeddings	Convert each chunk into a vector (list of numbers) capturing its meaning.	pending
7. Chroma vector store	Store all vectors + text in a local vector database for similarity search.	pending
8. SQLRecordManager	Track which chunks are already indexed, so re-running indexing doesn't create duplicates.	pending
9. Streamlit indexing UI	A web page to trigger/monitor indexing a chapter.	pending
10. Retriever + teacher prompt	Given a student's question, find the most relevant chunks and have the LLM explain them pedagogically.	pending

11. Conversation memory	Support natural follow-up questions across a chat.	pending
12. Streamlit chat UI	The actual chat page a student uses.	pending
13. DeepEval evaluation	Objectively score answer quality (faithfulness, relevancy).	pending
14. Polish	Error handling, logging, usage docs, deployment notes.	pending

---

8. Command cheat-sheet so far

```
# Run any Python file/module inside this project's environment
uv run python -m rag_ncert_biology_teacher.<module.path>

# Add a new library later
uv add <package-name>

# See what's changed since the last git commit
git status

# Save a checkpoint (only after you're told what's being committed)
git add <files>
git commit -m "message"
```

---

9. Mini glossary (plain-English definitions)

- **RAG (Retrieval-Augmented Generation):** search your own documents first, then have the AI answer using only what was found — reduces made-up facts.
- **Embedding:** a list of numbers representing the *meaning* of a piece of text, produced by a machine learning model. Similar meanings → similar numbers, even if the wording is completely different.
- **Vector database (Chroma):** a database specialized in storing embeddings and quickly finding "which stored items are most similar to this new item" — this is how semantic search works.
- **Chunk:** a small piece of a document (a paragraph-ish amount of text). We search over chunks, not whole books, so retrieved context stays focused.
- **Retriever:** the component that takes a question, embeds it, and fetches the most similar chunks from the vector database.
- **LLM (Large Language Model):** the AI model (e.g. Claude) that generates the final natural-language answer, using the retrieved chunks as context.

- **Prompt:** the instructions we hand the LLM, e.g. "you are a teacher, explain this concept to a student using only the following textbook excerpts."